

PLSQL Procedure Examples



write me all type of procedure structure and example in plsql oracle

Got it! Let's break down **PL/SQL procedures** in Oracle with **all typical structure styles**, their **syntax**, and **examples** for each.

1 Basic Procedure

Structure:

```
sql

CREATE [OR REPLACE] PROCEDURE procedure_name
IS
BEGIN
    -- Statements
END;
/
```

Example:

```
sql

CREATE OR REPLACE PROCEDURE say_hello
IS
BEGIN
    DBMS_OUTPUT.PUT_LINE('Hello, World!');
END;
/
```

Run:

```
sql

BEGIN
    say_hello;
END;
/
```

2 Procedure with Parameters

Structure:

```
sql

CREATE [OR REPLACE] PROCEDURE procedure_name (
    param1 IN datatype,
    param2 OUT datatype,
```

```

    param3 IN OUT datatype
)
IS
BEGIN
    -- Statements
END;
/

```

Example:

```

sql

CREATE OR REPLACE PROCEDURE add_numbers (
    num1 IN NUMBER,
    num2 IN NUMBER,
    sum OUT NUMBER
)
IS
BEGIN
    sum := num1 + num2;
END;
/

```

Run:

```

sql

DECLARE
    result NUMBER;
BEGIN
    add_numbers(5, 10, result);
    DBMS_OUTPUT.PUT_LINE('Sum is: ' || result);
END;
/

```

3 Procedure with EXCEPTION Handling

Structure:

```

sql

CREATE OR REPLACE PROCEDURE procedure_name
IS
BEGIN
    -- Statements
EXCEPTION
    WHEN exception_name THEN
        -- Handling
END;
/

```

Example:

```

sql

CREATE OR REPLACE PROCEDURE divide_numbers (
    num1 IN NUMBER,
    num2 IN NUMBER,
    result OUT NUMBER
)
IS
BEGIN

```

```

    result := num1 / num2;
EXCEPTION
    WHEN ZERO_DIVIDE THEN
        DBMS_OUTPUT.PUT_LINE('Division by zero is not allowed.');
```

```

    result := NULL;
END;
/
```

Run:

```

sql

DECLARE
    res NUMBER;
BEGIN
    divide_numbers(10, 0, res);
    DBMS_OUTPUT.PUT_LINE('Result: ' || res);
END;
/
```

4 Procedure with Local Variables

Structure:

```

sql

CREATE OR REPLACE PROCEDURE procedure_name
IS
    local_var datatype;
BEGIN
    -- Use local_var
END;
/
```

Example:

```

sql

CREATE OR REPLACE PROCEDURE find_square (
    num IN NUMBER
)
IS
    square NUMBER;
BEGIN
    square := num * num;
    DBMS_OUTPUT.PUT_LINE('Square is: ' || square);
END;
/
```

Run:

```

sql

BEGIN
    find_square(6);
END;
/
```

5 Procedure in a Package

Structure:

Package Specification:

```
sql

CREATE OR REPLACE PACKAGE my_package IS
    PROCEDURE greet(name IN VARCHAR2);
END my_package;
/
```

Package Body:

```
sql

CREATE OR REPLACE PACKAGE BODY my_package IS
    PROCEDURE greet(name IN VARCHAR2) IS
    BEGIN
        DBMS_OUTPUT.PUT_LINE('Hello, ' || name);
    END;
END my_package;
/
```

Run:

```
sql

BEGIN
    my_package.greet('Arham');
END;
/
```

6 Procedure with Cursor

Structure:

```
sql

CREATE OR REPLACE PROCEDURE procedure_name
IS
    CURSOR cursor_name IS SELECT ...;
BEGIN
    FOR rec IN cursor_name LOOP
        -- Process rec
    END LOOP;
END;
/
```

Example:

```
sql

CREATE OR REPLACE PROCEDURE list_employees
IS
    CURSOR emp_cur IS SELECT first_name, last_name FROM employees;
BEGIN
```

```
FOR emp_rec IN emp_cur LOOP
    DBMS_OUTPUT.PUT_LINE(emp_rec.first_name || ' ' || emp_rec.last_name);
END LOOP;
END;
/
```

Run:

```
sql

BEGIN
    list_employees;
END;
/
```

✓ Summary Cheat Sheet

Procedure Type	Key Feature	Use
Basic	No params	Simple tasks
With Parameters	IN/OUT/IN OUT	Input/Output
With Exception	EXCEPTION block	Handle errors
With Local Vars	DECLARE local vars	Intermediate data
In Package	Organized in a package	Modularity
With Cursor	Loop through query	Bulk processing

If you'd like, I can prepare **test table scripts** too — just say **“Yes, show me test tables!”** and I'll write them for you!



do the same for functions

Absolutely! Let's do the same for **PL/SQL functions** in Oracle: all the key **types**, **structures**, and **clear examples**.

✓ 1 Basic Function

Structure:

```
sql

CREATE [OR REPLACE] FUNCTION function_name
RETURN datatype
IS
BEGIN
    -- Statements
    RETURN value;
END;
/
```

Example:

```
sql

CREATE OR REPLACE FUNCTION get_pi
RETURN NUMBER
IS
BEGIN
    RETURN 3.1416;
END;
/
```

Run:

```
sql

BEGIN
    DBMS_OUTPUT.PUT_LINE('PI is: ' || get_pi);
END;
/
```

✓ 2 Function with Parameters

Structure:

```
sql

CREATE OR REPLACE FUNCTION function_name (
    param1 IN datatype,
    param2 IN datatype
)
RETURN datatype
IS
BEGIN
    -- Statements
    RETURN value;
END;
/
```

Example:

```
sql

CREATE OR REPLACE FUNCTION multiply (
    num1 IN NUMBER,
    num2 IN NUMBER
)
RETURN NUMBER
IS
BEGIN
    RETURN num1 * num2;
END;
/
```

Run:

```
sql

BEGIN
    DBMS_OUTPUT.PUT_LINE('Product: ' || multiply(4, 5));
END;
```

```
END;
/
```

✓ 3 Function with EXCEPTION Handling

Structure:

```
sql

CREATE OR REPLACE FUNCTION function_name (
    param IN datatype
)
RETURN datatype
IS
BEGIN
    -- Statements
    RETURN value;
EXCEPTION
    WHEN exception_name THEN
        -- Handle exception
        RETURN default_value;
END;
/
```

Example:

```
sql

CREATE OR REPLACE FUNCTION safe_divide (
    num1 IN NUMBER,
    num2 IN NUMBER
)
RETURN NUMBER
IS
BEGIN
    RETURN num1 / num2;
EXCEPTION
    WHEN ZERO_DIVIDE THEN
        DBMS_OUTPUT.PUT_LINE('Division by zero!');
        RETURN NULL;
END;
/
```

Run:

```
sql

BEGIN
    DBMS_OUTPUT.PUT_LINE('Result: ' || safe_divide(10, 0));
END;
/
```

✓ 4 Function with Local Variables

Structure:

```
sql

CREATE OR REPLACE FUNCTION function_name (
    param IN datatype
)
RETURN datatype
IS
    local_var datatype;
BEGIN
    -- Use local_var
    RETURN value;
END;
/
```

Example:

```
sql

CREATE OR REPLACE FUNCTION cube (
    num IN NUMBER
)
RETURN NUMBER
IS
    result NUMBER;
BEGIN
    result := num * num * num;
    RETURN result;
END;
/
```

Run:

```
sql

BEGIN
    DBMS_OUTPUT.PUT_LINE('Cube: ' || cube(3));
END;
/
```

Function in a Package

Structure:

Package Specification:

```
sql

CREATE OR REPLACE PACKAGE math_package IS
    FUNCTION square(num IN NUMBER) RETURN NUMBER;
END math_package;
/
```

Package Body:

```
sql

CREATE OR REPLACE PACKAGE BODY math_package IS
    FUNCTION square(num IN NUMBER) RETURN NUMBER IS
    BEGIN
        RETURN num * num;
    END;
END;
```



```
END;
END math_package;
/
```

Run:

```
sql

BEGIN
  DBMS_OUTPUT.PUT_LINE('Square: ' || math_package.square(7));
END;
/
```

Function with Cursor

Structure:

```
sql

CREATE OR REPLACE FUNCTION function_name
RETURN datatype
IS
  CURSOR cursor_name IS SELECT ...;
  total datatype := 0;
BEGIN
  FOR rec IN cursor_name LOOP
    -- Process rec
  END LOOP;
  RETURN total;
END;
/
```

Example:

```
sql

CREATE OR REPLACE FUNCTION count_employees
RETURN NUMBER
IS
  CURSOR emp_cur IS SELECT employee_id FROM employees;
  total NUMBER := 0;
BEGIN
  FOR emp_rec IN emp_cur LOOP
    total := total + 1;
  END LOOP;
  RETURN total;
END;
/
```

Run:

```
sql

BEGIN
  DBMS_OUTPUT.PUT_LINE('Total Employees: ' || count_employees);
END;
/
```

Calling Functions in SQL SELECT

Functions that **do not have OUT/IN OUT parameters** and **don't use DBMS_OUTPUT** can be called directly in **SELECT**.

Example:

```
sql

-- Function
CREATE OR REPLACE FUNCTION double_salary (
  sal IN NUMBER
)
RETURN NUMBER
IS
BEGIN
  RETURN sal * 2;
END;
/
```

Run in SELECT:

```
sql

SELECT employee_id, double_salary(salary) AS double_sal
FROM employees;
```

Key Differences: Procedure vs Function

Feature	Procedure	Function
Return value	Does not return directly	Must return a value
Call in SQL	Cannot be used in SELECT	Can be used in SELECT
Parameters	IN, OUT, IN OUT	Only IN (OUT is invalid)
Usage	Perform action	Return computed value

Summary Cheat Sheet

Function Type	Key Feature
Basic	Simple return
With Params	Input values
With Exception	Handle errors safely
With Local Vars	Intermediate calc
In Package	Modular design
With Cursor	Process multiple rows
In SELECT	Must return a value

If you'd like, I can also give you **sample tables** or **test data scripts** for these — just say **"Yes, show me!"** and I'll write them!



Give me theoretical tricky question and answers about functions in plsql

Alright — here's a **set of tricky, theory-focused PL/SQL function questions** that interviewers or exams often ask, **with clear answers**. These will test your **conceptual understanding**, not just syntax.

1 Can a PL/SQL function have OUT or IN OUT parameters?

? Question:

Is it valid to declare a PL/SQL function with OUT or IN OUT parameters? Why or why not?

✓ Answer:

No, a PL/SQL function **cannot** have OUT or IN OUT parameters.

A function **must** return exactly **one value** using the **RETURN** statement. Allowing OUT or IN OUT parameters would make it behave like a procedure and break its purpose of being used **in SQL expressions**.

2 Can a function perform DML (INSERT, UPDATE, DELETE)?

? Question:

Can you write a PL/SQL function that does an INSERT or UPDATE? Is there any restriction when you call it from SQL?

✓ Answer:

A function **can** perform DML statements **inside PL/SQL blocks** (like in a trigger or anonymous block). However, if you try to call that function **inside a SQL SELECT**, it will fail with **ORA-14551: cannot perform a DML operation inside a query**.

So:

- ✓ DML allowed inside PL/SQL contexts.
- ✗ DML not allowed in functions used in SQL SELECT statements.

3 Can you call DBMS_OUTPUT.PUT_LINE in a function that's used in SELECT?

? Question:

If you call **DBMS_OUTPUT.PUT_LINE** inside a function, and then use that function in a SELECT query, what happens?

✓ **Answer:**

DBMS_OUTPUT has **no effect** when used in SQL.

If you call a function with DBMS_OUTPUT.PUT_LINE in a SQL statement, you won't see any output — because DBMS_OUTPUT only works in PL/SQL blocks, not inside SQL engines.

So the function works, but the output won't display.

4 Can you have more than one RETURN statement?

? **Question:**

Is it valid to have multiple RETURN statements in a PL/SQL function?

✓ **Answer:**

Yes, you **can** have multiple RETURN statements in different branches (IF, CASE, LOOP exits).

But only **one** will execute at runtime.

Example:

```
sql

IF condition THEN
  RETURN 1;
ELSE
  RETURN 2;
END IF;
```

5 What happens if a function does not hit any RETURN statement?

? **Question:**

What if your function runs but no RETURN statement is executed — what error do you get?

✓ **Answer:**

Oracle raises PLS-00221: 'FUNCTION_NAME' must return a value.

The function must return a value in **every possible path**. Otherwise, it fails at runtime.

6 Can you call a procedure from inside a function?

? **Question:**

Is it valid to call a procedure from inside a function?

✓ **Answer:**

Yes, a function can **call a procedure**.

BUT — if that procedure does DML or DBMS_OUTPUT or commits/rollbacks, and you call the function inside a SQL SELECT, you'll get an error like ORA-14551.

So calling procedures is fine, but watch out for side effects if the function is used in SQL.

7 Can a function return a table?

? Question:

Can you write a PL/SQL function that returns a table?

✓ Answer:

Not exactly a table directly — but you can return:

- A **collection** (like a `TABLE OF NUMBER`).
- Or a **REF CURSOR** (pipelined functions).

These can then be used in **table functions** with `TABLE()` in SQL.

Example:

```
sql

FUNCTION get_numbers RETURN number_table_type PIPELINED IS ...
```

8 What is a deterministic function?

? Question:

What does the **DETERMINISTIC** keyword mean in a function? When would you use it?

✓ Answer:

DETERMINISTIC means the function always returns the **same output for the same input**.

This allows Oracle to safely **cache** results for performance.

Example: a pure math function like `square(num)` is deterministic.

You must declare **DETERMINISTIC** if you want to use the function in a **function-based index**.

9 Can you overload functions?

? Question:

Can you overload functions in PL/SQL?

✓ Answer:

Yes, in a **package** or **object type**, you can declare multiple functions with the **same name** but different parameter lists.

Overloading is not possible for standalone functions.

10 What is the difference between RETURN in functions vs EXIT?

? Question:

In a function, what does **RETURN** do vs **EXIT**?

✓ **Answer:**

- **RETURN** exits the function and returns a value to the caller.
- **EXIT** is only used to **exit a loop**, not the whole function.

⚡ Summary: Key tricky points

- ✓ Functions **must** return a value.
- ✓ They **can't have OUT/IN OUT** params.
- ✓ They **can't do DML** if called in a SQL SELECT.
- ✓ **DBMS_OUTPUT** doesn't work in SELECT.
- ✓ Use **DETERMINISTIC** for caching/indexing.
- ✓ Overload only inside **packages** or **types**.

If you'd like, I can make you a **mini quiz sheet** or a **PDF** to test yourself — just say **"Yes, make the quiz!"**



give me the difference between user defined function and built in function in plsql

Great question — here's a clear **comparison** between **user-defined functions** and **built-in functions** in **PL/SQL** (Oracle):

✓ 1 What they are

Aspect	User-Defined Function	Built-In Function
Definition	A function you write yourself using CREATE FUNCTION .	A function provided by Oracle as part of SQL or PL/SQL.
Example	<code>get_bonus(salary NUMBER) RETURN NUMBER</code>	<code>SYSDATE, TO_CHAR, SUM(), ROUND(), LENGTH()</code>

✓ 2 Where they reside

Aspect	User-Defined Function	Built-In Function
Stored in	Database schema (stored as an object).	Part of Oracle engine — always available.
Ownership	Owned by user who created it.	Owned by Oracle system.

✓ 3 How you call them

Aspect	User-Defined Function	Built-In Function
How to use	Call in PL/SQL blocks, or SQL if allowed.	Call in SQL, PL/SQL, triggers — always valid.
Usage in SELECT	Only if it has no side effects (no DML).	Always valid (designed to be safe).

✓ 4 Examples

Aspect	Example
User-Defined	<code>sql CREATE FUNCTION calc_tax(p_salary NUMBER) RETURN NUMBER IS BEGIN RETURN p_salary * 0.1; END;</code>
Built-In	<code>sql SELECT ROUND(123.456, 2) FROM dual;</code>

✓ 5 Flexibility

Aspect	User-Defined Function	Built-In Function
Custom Logic	Any custom business rules.	Only predefined operations.
Extendability	You can create as many as needed.	You cannot change Oracle's built-ins.

✓ 6 Performance & Caching

Aspect	User-Defined Function	Built-In Function
Performance	Depends on how you write it.	Highly optimized and tested by Oracle.
Deterministic	You must declare it DETERMINISTIC if it always returns same output for same input — to use in indexes.	Many built-ins are already deterministic internally.

✓ 7 Examples in practice

- **Built-In:** `SYSDATE`, `TO_DATE()`, `NVL()`, `LENGTH()`, `SUBSTR()`, `ABS()`, `SUM()`, `AVG()`
- **User-Defined:** `get_discount(price NUMBER)`, `calc_gpa(marks NUMBER)`, `format_name(first VARCHAR2, last VARCHAR2)`

⚡ Final key difference

User-Defined Functions 🖱 For **custom logic** specific to your business.

Built-In Functions 🖱 For **general, commonly used operations** Oracle provides for everyone.

If you'd like, I can also give you **practice questions** comparing them, or a **sample script** that shows both together in one query — just say **"Yes, give me that too!"** 🚀



```
BEGIN
  UPDATE EMP
  SET SALARY = SALARY * 1.25
  WHERE ID = 101;

  IF (SQL%IS_OPEN) THEN
    DBMS_OUTPUT.PUT_LINE('SQL IS FUN');
  END IF;
END;
/
```

explain if SQL is fun will print or not and why

Alright — let's break this **PL/SQL block** down carefully:

✓ The Code

```
sql

BEGIN
  UPDATE EMP
  SET SALARY = SALARY * 1.25
  WHERE ID = 101;

  IF (SQL%IS_OPEN) THEN
    DBMS_OUTPUT.PUT_LINE('SQL IS FUN');
  END IF;
END;
/
```

✓ What's happening?

- 1 The **UPDATE** statement runs — it's a **SQL DML statement**, not a cursor.
- 2 Then you check:

```
sql

IF (SQL%IS_OPEN) THEN
  ...
END IF;
```

✓ What does SQL%IS_OPEN mean?

- SQL is an **implicit cursor**.
- It's a *system-managed* cursor that automatically handles **DML statements** (**SELECT INTO**, **INSERT**, **UPDATE**, **DELETE**).
- **SQL%IS_OPEN** tells you whether the implicit cursor is open.

BUT — **implicit cursors are never explicitly opened or closed by you.**

They **open automatically**, run, and then **close automatically**.

So after the **UPDATE** finishes, **SQL%IS_OPEN** is always **FALSE** — because the cursor is automatically closed when the statement completes.

✓ So, will SQL IS FUN print?

No.

Reason:

The **IF** condition **SQL%IS_OPEN** evaluates to **FALSE** ⇒ so **DBMS_OUTPUT.PUT_LINE('SQL IS FUN');** is **never executed**.

✓ What would work instead?

If you want to know if rows were affected by the **UPDATE**, you could check **SQL%FOUND** or **SQL%ROWCOUNT**.
Example:

```
sql

BEGIN
  UPDATE EMP
  SET SALARY = SALARY * 1.25
  WHERE ID = 101;

  IF SQL%FOUND THEN
    DBMS_OUTPUT.PUT_LINE('SQL IS FUN');
  END IF;
END;
/
```

This works, because **SQL%FOUND** is **TRUE** if **at least one row** was affected.

🔑 Conclusion

Check	Result
SQL%IS_OPEN	FALSE for implicit cursor — so won't print.
SQL%FOUND	TRUE if rows affected — so will print.

Let me know if you'd like a **quick demo with a table and test rows** — I can write that for you too! ✓ ✨